CS503/AI211 - Machine Learning
Practice Sheet

Assigned on 17-04-2026

Question: K-Means Clustering

You have given 8 data points:

Index	x_1	x_2
1	1	1
2	2	2
3	2	3
4	4	3
5	4	4
6	4	5
7	6	5
8	5	6

1. Taking $k = 2$, with starting points for the centroids of the cluster as $(0, 0)$ and $(6, 6)$, perform 2 iterations of k-means clustering on the given data points.
2. Suppose your initial centroids are now $(3, 4)$ and $(6, 1)$. What do you think the outcome of clustering will be? What does this tell you about the dependence of k-means on the starting points? Is this a strength or a drawback of k-means?
3. Define the silhouette score and write its formula. What is its range? What does a score of:
   - $+1$
   - 0
   - -1

   mean?

Question: MLP Design

Consider a 2D classification problem where a polygon-shaped decision region is formed using an MLP.

1. The region is defined by the intersection of the following 3 linear half-spaces:
   - $x_1 \leq 1 \Rightarrow -x_1 + 1 \geq 0$

- $x_2 \leq 1 \Rightarrow -x_2 + 1 \geq 0$
- $x_1 + x_2 \geq -1 \Rightarrow x_1 + x_2 + 1 \geq 0$

2. Each boundary is implemented by a hidden neuron with a step activation:

$$\text{Step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$$

Construct the weight matrix $W^{(1)}$ and bias vector $b^{(1)}$ such that:

$$a^{(1)} = \text{Step}(W^{(1)}x + b^{(1)})$$

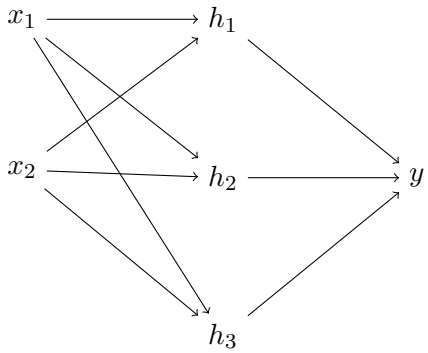
3. Design $W^{(2)}$ and $b^{(2)}$ such that $\hat{y} = 1$ if and only if the point lies in the intersection of all three regions.

4. Plot the boundaries and shade the resulting decision region.

Question: Forward Pass in Artificial Neural Network

Consider a neural network with the following architecture:

Input: $x \in \mathbb{R}^2$
Hidden layer: 3 neurons with ReLU activation
Output layer: 1 neuron with no activation



$$W_1 = \begin{bmatrix} 1 & -1 \\ 0 & 2 \\ -2 & 1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

$$W_2 = [2 \ -1 \ 1], \quad b_2 = 0$$

1. Write the output of the network y in matrix form using ReLU activation.

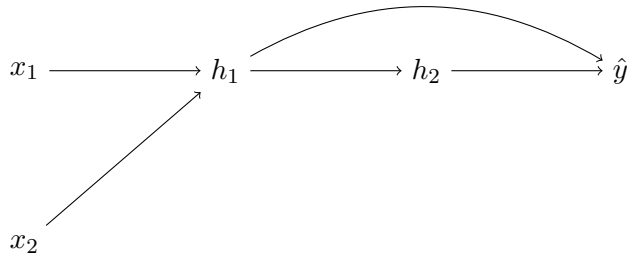
2. Now suppose the ReLU activation is replaced by the identity function. Simplify the expression for y and show that it reduces to a linear function of x.

Question: Backpropagation in ANN

Consider a Feedforward Neural Network with a branching architecture:

Input Layer: $x = [x_1, x_2]^T$
Hidden Layer 1: One neuron h_1 (ReLU activation)
Hidden Layer 2: One neuron h_2 (ReLU activation)
Output Layer: One neuron $\hat{y}$ (Sigmoid activation)

Special Architecture: The output of h_1 is fed into both h_2 and the final output neuron $\hat{y}$.



Parameters:

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b^{(1)} = 0$$

$$w^{(2)} = 1, \quad b^{(2)} = 0 \quad \text{(Weight from } h_1 \text{ to } h_2\text{)}$$

$$V = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad b^{(3)} = 0 \quad \text{(Weights from } h_1 \text{ and } h_2 \text{ to output)}$$

1. Compute the values for $z^{(1)}, h_1, z^{(2)}, h_2, z^{(3)}$, and $\hat{y}$.
2. Given the target $y = 1$ and Binary Cross-Entropy loss, compute:
   (a) $\delta^{(3)} = \frac{\partial L}{\partial z^{(3)}}$
   (b) $\frac{\partial L}{\partial h_2}$ and $\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}}$
   (c) $\frac{\partial L}{\partial h_1}$
   (Note: Sum gradients from both the h_2 path and the direct output path)
   (d) $\delta^{(1)} = \frac{\partial L}{\partial z^{(1)}}$

Question: Convolutional Neural Networks

1. Consider a CNN with the following architecture:
   Input: $32 \times 32 \times 3$

   (i) Conv1: 3×3, 16 filters, stride = 1, padding = 1
   ReLU

(ii) MaxPool: 2×2, stride $= 2$

(iii) Conv2: 3×3, 32 filters, stride $= 1$, padding $= 1$
ReLU

(iv) MaxPool: 2×2, stride $= 2$

(v) Flatten

(vi) Dense: 128 neurons
ReLU

(vii) Dense: 10 neurons
Softmax

For each layer (marked (i) to (vii)), calculate:

(a) Number of weights

(b) Number of biases

(c) Output shape

2. You are given the following activation map:

$$X = \begin{bmatrix} 1 & 3 & 2 & 4 & 6 & 5 \\ 5 & 6 & 1 & 2 & 3 & 4 \\ 7 & 2 & 8 & 1 & 0 & 3 \\ 4 & 5 & 9 & 2 & 1 & 6 \\ 3 & 1 & 2 & 7 & 8 & 9 \\ 6 & 4 & 5 & 3 & 2 & 1 \end{bmatrix}$$

Apply the following operations on X:

(a) Max pooling $(2 \times 2,\ \text{stride} = 2)$

(b) Average pooling $(2 \times 2,\ \text{stride} = 2)$

3. (MCQ) If all the weights of a filter are zero, but the bias is non-zero, what happens?

(a) Always outputs zero

(b) Learns slowly

(c) Depends on input

(d) Outputs a constant value equal to the bias

(e) Detects uniform regions

4. Why are CNNs more parameter-efficient than fully connected layers?

Question: Convolution I

1. Consider the following filter:

$$K_1 = \begin{bmatrix} -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \end{bmatrix}$$

Consider the following two patches:

$$X_A = \begin{bmatrix} 2 & 3 & 7 & 8 \\ 2 & 2 & 8 & 9 \\ 1 & 3 & 7 & 9 \\ 2 & 2 & 8 & 8 \end{bmatrix} \quad X_B = \begin{bmatrix} 5 & 6 & 4 & 5 \\ 5 & 5 & 6 & 4 \\ 4 & 5 & 5 & 6 \\ 6 & 4 & 5 & 5 \end{bmatrix}$$

Convolve the filter K_1 with the two patches X_A and X_B. You should obtain a single scalar value for each (say a and b).

2. Consider another filter:

$$K_2 = \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{bmatrix}$$

Which one of K_1 and K_2:

(a) is a line filter?

(b) is an edge filter?

Explain your conclusion intuitively. (Hint: If you look at an image, what would you call a line, and what would you call an edge? The values in K_1 and K_2 can provide hints.)

3. Using the values a and b from part (1), and your intuition from part (2), which of X_A and X_B is a flat surface, and which contains a line/edge?

Question: Convolution II

Consider a convolutional neural network (CNN) trained for image classification on a dataset of handwritten digits (e.g., digits 0–9). The training dataset contains images where all digits are upright (no rotation). (These are intuitive questions: only knowledge of the convolution operation is required.)

1. Is a standard CNN rotation invariant or rotation variant? Why?

2. Consider an input image x and its rotated version x_r (rotated by 90°).

   (a) Will the feature maps produced by the CNN for x and x_r be the same?

   (b) Justify your answer based on how convolutional filters operate.

3. Suppose the trained CNN achieves high accuracy ($\approx 98\%$) on the test set containing upright digits. Now, the same test images are rotated by 90°.

(a) Predict how the model's performance will change.

(b) Provide reasoning in terms of learned filters and spatial patterns.

4. To address the issue above, the training dataset is augmented with rotated versions of each image (e.g., $0°, 90°, 180°, 270°$).

   (a) Explain how this augmentation helps improve model performance on rotated images.

   (b) Does this make the CNN truly rotation invariant, or does it approximate invariance? Explain.

5. (Optional question, out-of-syllabus, for extra understanding of Computer Vision.) Even after augmentation, the CNN may still struggle with arbitrary rotation angles (e.g., $37°$).

   (a) Why does this limitation occur?

   (b) Suggest one architectural modification or alternative approach that can inherently handle rotations better, and briefly explain how it works.

Question: Optimization

1. A company is training a deep learning model on a dataset containing **50 million samples**. During experimentation:

   - Using **Batch Gradient Descent**, training is very slow and memory usage is extremely high.
   - Using **Stochastic Gradient Descent**, training becomes faster but the loss fluctuates heavily.
   - Using **Mini-Batch Gradient Descent**, training is stable and reasonably fast.

   Based on this scenario, answer the following:

   (a) Explain why Batch Gradient Descent performs poorly for this dataset.

   (b) Analyze the reason behind fluctuations in Stochastic Gradient Descent.

   (c) Justify why Mini-Batch Gradient Descent provides a better balance.

   (d) Recommend the most suitable method and explain your choice in terms of:

      - Convergence behavior
      - Computational efficiency
      - Scalability

2. A machine learning engineer is developing models for three different applications:

   - **Application A:** A small dataset that easily fits into memory, where high accuracy and stable convergence are critical.
   - **Application B:** A real-time recommendation system that requires fast updates and can handle noisy gradients.
   - **Application C:** A large-scale image classification task with millions of samples, where memory is limited but efficient training is required.

The engineer must choose between **Batch Gradient Descent**, **Stochastic Gradient Descent**, and **Mini-Batch Gradient Descent**.

Based on this scenario, answer the following:

(a) Analyze the trade-offs between:
- Accuracy
- Memory usage
- Update speed

for each gradient descent variant.

(b) Identify the most suitable optimization method for each application (A, B, and C).

(c) Justify your choices based on:
- Convergence behavior
- Computational efficiency
- Scalability

Question: Recurrent Neural Networks (RNN)

Part 1: Forward Pass (Numerical)

Given the input sequence:

$$x_1 = [1, 0], \quad x_2 = [0, 1], \quad x_3 = [1, 1]$$

and initial hidden state:

$$h_0 = [0, 0]$$

All vectors are in $\mathbb{R}^2$.

Weights:

$$W_x = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad W_h = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$$

Activation function:

$$\tanh$$

Assume identity output layer:

$$y_t = h_t$$

Assume target outputs:

$$y_1^{\text{true}} = [1, 0], \quad y_2^{\text{true}} = [0, 1], \quad y_3^{\text{true}} = [1, 1]$$

1. Compute h_1, h_2, h_3 using:
$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$
2. Compute outputs:
$$y_t = h_t$$
3. Compute squared loss:
$$L_t = \frac{1}{2} \| y_t - y_t^{\text{true}} \|_2^2$$
4. Compute total loss:
$$L = \sum_{t=1}^{3} L_t$$

Part 2: Backpropagation Through Time (BPTT)

1. Compute:
$$\frac{\partial L_t}{\partial h_t} = (h_t - y_t^{\text{true}})$$

2. Define the error term δ_t as:
$$\delta_t = \left(\frac{\partial L_t}{\partial h_t} + W_h^T \delta_{t+1}\right) \odot (1 - h_t^2)$$

   with base case:
$$\delta_3 = \frac{\partial L_3}{\partial h_3} \odot (1 - h_3^2)$$

3. Compute δ_3, then recursively compute δ_2 and δ_1.

4. Explain how gradients propagate through time using the recurrence relation above.

5. Show the vanishing gradient effect using:
$$(W_h^T)^2, \quad (W_h^T)^3$$
   and explain what happens when eigenvalues of W_h are less than 1.

Part 3: Gradient Computation

1. Compute $\delta_3, \delta_2, \delta_1$ explicitly.

2. Compute gradients with respect to recurrent weights:
$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^{3} \delta_t \, h_{t-1}^T$$

3. Compute gradients with respect to input weights:
$$\frac{\partial L}{\partial W_x} = \sum_{t=1}^{3} \delta_t \, x_t^T$$